

Москва, 2026

Guardrail Service for Conversational AI

Технический отчёт

Сервис модерации ответов conversational AI

Тестовое задание: проектирование и реализация
production-style MVP на Python / FastAPI

Автор: Backend Engineer

Стек: Python, FastAPI, Transformers, ChromaDB, OpenRouter

Дата: 18 мая 2026 г.

Оглавление

1	Постановка задачи	1
1.1	Описание проблемы	1
1.2	Зачем нужны guardrails для LLM	1
1.3	Категории рисков	1
1.4	Требования технического задания	2
2	Архитектура решения	3
2.1	Общая схема сервиса	3
2.2	Слой FastAPI API	3
2.3	Пайплайн локальной модерации	4
2.4	Пайплайн облачной модерации	4
2.5	RAG и ChromaDB	4
2.6	Интеграция OpenRouter / OpenAI	4
2.7	Логирование и observability	4
3	Локальный подход (OSS moderation)	5
3.1	Модель toxic-bert	5
3.2	Regex PII detector	5
3.3	Regex jailbreak detection	5
3.4	Семантический детектор jailbreak	5
3.5	Risk scorer и ensemble aggregation	5
3.6	Сводка: плюсы и минусы локального пайплайна	6
4	Облачный подход (LLM + RAG)	7
4.1	Retrieval pipeline	7
4.2	Embeddings	7
4.3	Policy chunks	7
4.4	Structured JSON classification	7
4.5	Retry logic	7
4.6	Fallback strategy	7
4.7	Плюсы и минусы облачного пайплайна	8
5	Semantic Jailbreak Detection	9
5.1	Назначение	9
5.2	sentence-transformers	9
5.3	Cosine similarity	9
5.4	Canonical jailbreak prompts	9
5.5	Threshold tuning	9
5.6	Отличие от regex	10

6	API и интерфейсы	11
6.1	Эндпоинты	11
6.2	Схема запроса и ответа	11
6.3	Healthcheck	11
7	Benchmark и сравнение подходов	12
7.1	Результаты	12
7.2	Trade-off анализ	12
8	Production-рекомендация	13
8.1	Local как основной контур	13
8.2	Cloud как secondary layer	13
8.3	Гибридная архитектура	13
9	Соответствие требованиям 152-ФЗ	14
9.1	Риски облачного LLM	14
9.2	Data residency	14
9.3	Логирование и РП	14
10	Масштабирование и production concerns	15
10.1	Целевая нагрузка: 2 млн сообщений/час	15
10.2	Стратегии	15
11	Тестирование	16
12	Заключение	17
12.1	Реализовано	17
12.2	Направления улучшения	17
12.3	Итоговая рекомендация	17

Глава 1

Постановка задачи

1.1. Описание проблемы

Корпоративные и потребительские системы на базе больших языковых моделей (LLM) генерируют ответы в режиме реального времени. Даже при наличии системных промптов модель может выдавать токсичный контент, персональные данные, инструкции по причинению вреда или поддаваться атакам jailbreak / prompt injection. После генерации ответа требуется независимый контур проверки --- *guardrail* --- который возвращает структурированное решение для шлюза доставки сообщения пользователю.

Guardrail Service --- HTTP-сервис, принимающий текст ответа ассистента (и опционально контекст диалога) и возвращающий решение модерации в машиночитаемом JSON.

1.2. Зачем нужны guardrails для LLM

- **Безопасность пользователей** --- блокировка self-harm, насилия, мошенничества.
- **Комплаенс** --- снижение регуляторных рисков (в т. ч. 152-ФЗ при обработке ПДн).
- **Репутационные риски** --- предотвращение утечки РИ и токсичных формулировок.
- **Устойчивость к атакам** --- детекция попыток обхода политик (DAN, ignore instructions).
- **Аудит** --- объяснимые решения с категорией и уровнем риска.

1.3. Категории рисков

Сервис поддерживает 18 категорий нарушений политики и категорию none:

Категория	Смысл
hate_speech	Разжигание ненависти, дискриминация
extremism	Экстремизм, радикализация
violence	Насилие, угрозы
self_harm	Суицид, самоповреждение
medical_advice	Неквалифицированные мед. рекомендации
legal_advice	Юридические консультации вместо юриста
pii_leakage	Утечка персональных / секретных данных
sexual_content	Неприемлемый сексуальный контент
harassment	Травля, домогательства

Категория	Смысл
fraud	Мошенничество, фишинг
malware	Вредоносное ПО, эксплойты
jailbreak_attempts	Обход ограничений модели
prompt_injection	Инъекции в промпт
political_manipulation	Манипуляции выборами
disinformation	Дезинформация
toxic_language	Токсичная лексика
privacy_violations	Нарушение приватности
unsafe_instructions	Опасные пошаговые инструкции

Уровни риска: LOW, MEDIUM, HIGH. Решения: ALLOW, WARN, BLOCK.

1.4. Требования технического задания

1. REST API на FastAPI со Swagger-документацией.
2. Два независимых пайплайна модерации: локальный (OSS) и облачный (LLM + RAG).
3. Локально: transformer-модель, regex PII, правила jailbreak, risk scoring.
4. Облачно: ChromaDB, embeddings, RAG, LLM-классификация с JSON-ответом.
5. Docker, логирование, модульная архитектура, benchmark, pytest, README.
6. Набор политик (policy dataset) для RAG.

Глава 2

Архитектура решения

2.1. Общая схема сервиса

На рисунке 2.1 показана высокоуровневая архитектура.

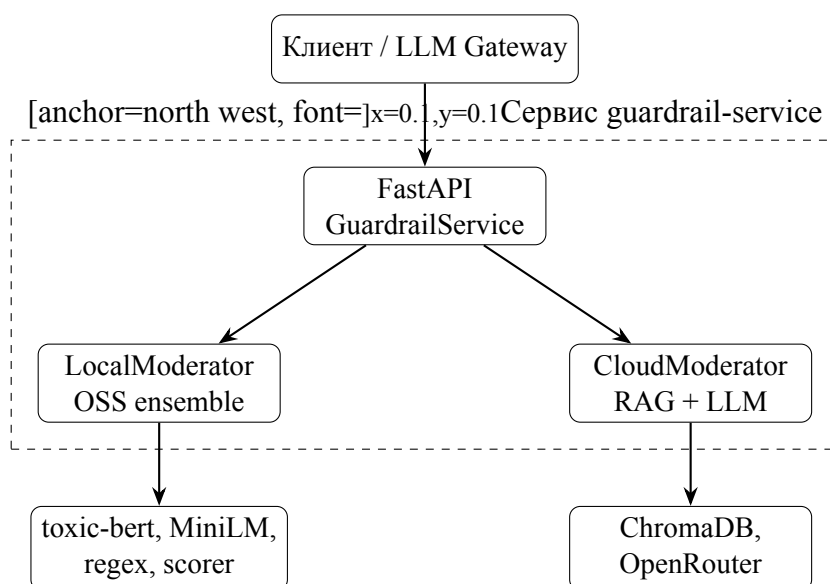


Рис. 2.1: Контекстная архитектура Guardrail Service

2.2. Слой FastAPI API

- `app/main.py` --- приложение, CORS, lifespan, маршрутизация `/api/v1`.
- `app/api/routes.py` --- эндпоинты модерации и `health`.
- `app/api/schemas.py` --- Pydantic-схемы запроса/ответа.
- `app/services/guardrail_service.py` --- фасад выбора пайплайна.

Валидация входа выполняется на уровне Pydantic (`content: 1--32000` символов). Ошибки бизнес-логики мапятся в HTTP 400/500 с логированием через `structlog`.

2.3. Пайплайн локальной модерации

Последовательность обработки в LocalModerator:

1. Классификация токсичности (unitary/toxic-bert).
2. Regex-детектор PII и паттернов jailbreak.
3. Семантический детектор jailbreak (MiniLM, cosine similarity).
4. Эвристики по ключевым словам (self-harm, malware, fraud и др.).
5. Агрегация сигналов в RiskScorer.

2.4. Пайплайн облачной модерации

1. Формирование запроса: content + context.
2. RAG: поиск top-K фрагментов политики в ChromaDB.
3. LLM: классификация с подстановкой политик в промпт.
4. Нормализация JSON в доменные enum'ы.

2.5. RAG и ChromaDB

Корпус: data/policies/policy_chunks.json (18 документов). Эмбединги: sentence-transformers/all-MiniLM-L6-v2. Хранилище: персистентный PersistentClient (CHROMA_PERSIST_DIR). Индексация: scripts/ingest_policies.py.

2.6. Интеграция OpenRouter / OpenAI

Клиент AsyncOpenAI с параметрами:

- OPENAI_API_KEY --- ключ OpenRouter или OpenAI;
- OPENAI_BASE_URL --- по умолчанию OpenAI, для OpenRouter: https://openrouter.ai/api
- OPENAI_MODEL --- идентификатор модели (напр. openai/gpt-4o-mini).

Ответ запрашивается в формате json_object, temperature=0.

2.7. Логирование и observability

Используется structlog:

- development --- цветной консольный вывод;
- production (APP_ENV=production) --- JSON-логи для агрегации.

Ключевые события: moderation_request, local_moderation_complete, semantic_jailbreak_request, llm_classification_complete, risk_aggregation_complete.

Глава 3

Локальный подход (OSS moderation)

3.1. Модель toxic-bert

Модель `unitary/toxic-bert` (HuggingFace pipeline, text-classification) возвращает многометочные скоры: `toxic`, `severe_toxic`, `obscene`, `threat`, `insult`, `identity_hate`. Порог `LOCAL_MODEL_THRESHOLD` (по умолчанию 0.65) определяет флаг нарушения. Маппинг меток на категории риска выполняется в `transformer.py`.

Плюсы: автономность, низкая стоимость inference после прогрева. **Минусы:** слабая чувствительность к jailbreak без явных токсичных маркеров; ограничение длины входа (~512 токенов).

3.2. Regex PII detector

`PIIDetector` ищет шаблоны: email, телефон (US), SSN, номер карты, IPv4, API-ключи (`sk-...`), AWS Access Key и др. При обнаружении формируется сигнал `pii_leakage` или `privacy_violations`.

3.3. Regex jailbreak detection

Набор `JAILBREAK_PATTERNS`: ignore instructions, DAN, role override, утечка system prompt, delimiter injection, policy bypass и др. Даёт высокую точность на известных формулировках при низкой латентности.

3.4. Семантический детектор jailbreak

Подробно --- глава 5. В локальном пайплайне добавляет сигнал `semantic_jailbreak` с категорией `jailbreak_attempts`.

3.5. Risk scorer и ensemble aggregation

Каждый детектор порождает `RiskSignal(source, category, score, detail)`. Веса категорий заданы в `CATEGORY_WEIGHTS` (`jailbreak` --- 1.0, `toxic_language` --- 0.6). Ar-

регированный скор:

$$s_{\text{ens}} = \min \left(\frac{1}{n} \sum_{i=1}^n w_i \cdot s_i, 1 \right)$$

Уровень риска: $s \geq 0.75 \Rightarrow \text{HIGH}$, $s \geq 0.45 \Rightarrow \text{MEDIUM}$. Решение: HIGH→BLOCK, MEDIUM→WARN, LOW→ALLOW.

3.6. Сводка: плюсы и минусы локального пайплайна

Таблица 3.1: Локальный пайплайн: оценка

Преимущества	Нет egress в облако LLM; предсказуемая латентность после warm-up; объяснимые сигналы (source + detail); выше ассигасу на тестовом бенч-марке.
Недостатки	Требует RAM/CPU для моделей; пороги требуют калибровки; слабее на редких перефразированиях без семантики; нет ссылок на policy ID.

Глава 4

Облачный подход (LLM + RAG)

4.1. Retrieval pipeline

```
# PolicyRAG.retrieve(query) -> list[dict]
results = collection.query(query_texts=[query], n_results=top_k)
```

Запрос --- конкатенация context и content. Метаданные чанков: category, risk_level, title.

4.2. Embeddings

Единая модель all-MiniLM-L6-v2 для ChromaDB и семантического jailbreak. Размерность 384, cosine distance в HNSW-индексе Chroma.

4.3. Policy chunks

18 JSON-объектов с полями id, title, category, risk_level, text. Покрывают все категории ТЗ.

4.4. Structured JSON classification

Системный промпт фиксирует схему ответа LLM. Пример полей: decision, category, risk_level, explanation. Парсинг через `json.loads`, нормализация в `_normalize()`.

4.5. Retry logic

Декоратор `tenacity`: до 3 попыток, экспоненциальная задержка 1--8 с. Снижает влияние сетевых сбоев OpenRouter.

4.6. Fallback strategy

При ошибке парсинга JSON: `decision=WARN, category=none, risk_level=MEDIUM` --- консервативная стратегия (не пропускать сомнительный контент молча).

4.7. Плюсы и минусы облачного пайплайна

Таблица 4.1: Облачный пайплайн: оценка

Преимущества	Гибкость формулировок политики; естественные объяснения; быстрое обновление политик через re-ingest без деплоя правил.
Недостатки	Зависимость от API; выше латентность и стоимость; риски трансграничной передачи данных; на бенчмарке accuracy 5/10.

Глава 5

Semantic Jailbreak Detection

5.1. Назначение

Regex ловит точные и близкие шаблоны, но пропускает перефразирования ("обойди ограничения безопасности другими словами"). Семантический модуль сопоставляет вход с *каноническими* jailbreak-примерами в пространстве эмбедингов.

5.2. sentence-transformers

Модель `SemanticJailbreakDetector` загружает `SentenceTransformer` один раз на процесс (`@lru_cache`). Кодирование выполняется под `torch.no_grad()`.

5.3. Cosine similarity

Векторы L2-нормализуются (`normalize_embeddings=True`). Косинусное сходство реализовано скалярным произведением:

$$\cos(\mathbf{q}, \mathbf{c}_i) = \mathbf{q}^\top \mathbf{c}_i$$

Берётся $\max_i \cos(\mathbf{q}, \mathbf{c}_i)$; при $\max \geq \tau$ --- детекция.

5.4. Canonical jailbreak prompts

23 эталонных фразы (DAN, ignore instructions, grandma exploit, STAN mode и др.) хранятся в `CANONICAL_JAILBREAKS`. Эмбединги кэшируются при первом вызове `_encode_canonical`

5.5. Threshold tuning

Параметр `SEMANTIC_JAILBREAK_THRESHOLD` (по умолчанию 0.62). Повышение τ снижает false positive, но увеличивает false negative. Рекомендуется калибровка на валидационной выборке заказчика.

5.6. Отличие от regex

Таблица 5.1: Regex vs семантический детектор

	Regex	Semantic
Механизм	Подстрока / паттерн	Сходство в embedding space
Перефразы	Слабо	Сильнее
Ложные срабатывания	На случайные совпадения	На близкие по смыслу безопасные фразы
Латентность	Микросекунды	Десятки--сотни мс (encode)

Интерфейс: `detect(text) -> (bool, float, str|None)` --- флаг, скор, matched example.

Глава 6

API и интерфейсы

6.1. Эндпоинты

Таблица 6.1: REST API

Метод	Путь	Назначение
GET	/api/v1/health	Статус и версия
POST	/api/v1/moderate	Модерация (approach в теле)
POST	/api/v1/moderate/local	Принудительно local
POST	/api/v1/moderate/cloud	Принудительно cloud
GET	/docs	Swagger UI

6.2. Схема запроса и ответа

```
{
  "content": Текст" ответаассистента ",
  "approach": "local",
  "context": Опциональный" контекстдиалога "
}
```

```
{
  "decision": "BLOCK",
  "category": "jailbreak_attempts",
  "risk_level": "HIGH",
  "explanation": "Ensemble moderation detected...",
  "approach": "local",
  "confidence": 0.87,
  "retrieved_policies": null
}
```

6.3. Healthcheck

GET /api/v1/health возвращает status, version, список approaches. Используется в Docker HEALTHCHECK.

Глава 7

Benchmark и сравнение подходов

Датасет: `tests/benchmark_dataset.json`, 10 размеченных примеров. Скрипт: `scripts/ben`

7.1. Результаты

Таблица 7.1: Сравнение пайплайнов на тестовом наборе

Метрика	Local (OSS)	Cloud (LLM+RAG)
Ассурасу	7 / 10 (70%)	5 / 10 (50%)
Средняя латентность	1402 ms	3515 ms

7.2. Trade-off анализ

Критерий	Local	Cloud
Латентность	Ниже (после warm-up)	Выше (сеть + LLM)
Стоимость	Только compute	Токены + API
Explainability	Источник сигнала (transformer, regex, semantic)	Текст LLM + policy IDs
Jailbreak robustness	Regex + semantic + rules	Зависит от промпта и RAG
Масштабируемость	Горизонтально, stateless*	Rate limits провайдера

*Модели в памяти процесса; для 2M msg/h нужен пул воркеров.

Глава 8

Production-рекомендация

8.1. Local как основной контур

На основании бенчмарка и архитектурных свойств **рекомендуется** использовать локальный пайплайн для inline-модерации каждого ответа:

- выше ассурасу на фиксированном наборе;
- ниже латентность;
- отсутствие передачи текста во внешний LLM.

8.2. Cloud как secondary layer

Облачный пайплайн целесообразен для:

- случаев WARN (второе мнение);
- асинхронной очереди human/LLM review;
- сценариев, где критична ссылка на текст корпоративной политики.

8.3. Гибридная архитектура

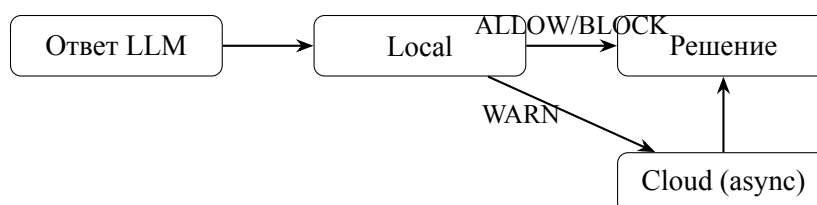


Рис. 8.1: Рекомендуемый гибридный контур

Глава 9

Соответствие требованиям 152-ФЗ

Федеральный закон № 152-ФЗ регулирует обработку персональных данных (ПДн) граждан РФ. Модерация может обрабатывать ПДн из пользовательских сообщений.

9.1. Риски облачного LLM

Передача текста в OpenRouter/зарубежного провайдера может constitute трансграничную передачу ПДн. Требуется правовое основание, уведомление РКН, договор с оператором, оценка страны получателя.

9.2. Data residency

Для контуров с требованием локализации --- размещение сервиса и логов в РФ, **отключение cloud moderation** для потоков с ПДн, использование только local pipeline.

9.3. Логирование и РИ

`structlog` не должен писать полный пользовательский текст на INFO в prod. Рекомендации: маскирование, хеширование, сокращённый preview, отдельный audit storage с контролем доступа и сроком хранения.

Глава 10

Масштабирование и production concerns

10.1. Целевая нагрузка: 2 млн сообщений/час

≈ 556 RPS среднее, пики ×3--5. Один синхронный запрос local ~50--200 ms после warm-up (бенчмарк 1402 ms включает cold start моделей).

10.2. Стратегии

- **Horizontal scaling** --- N реплик FastAPI за балансировщиком.
- **Model warmup** --- прогрев toxic-bert и MiniLM при старте пода.
- **Caching** --- кэш эмбедингов канонических jailbreak; опционально LRU по hash(content).
- **Async processing** --- WARN/BLOCK review через очередь (Kafka/RabbitMQ).
- **Rate limiting** --- на API gateway; защита от abuse.
- **Observability** --- метрики Prometheus: moderation_latency_seconds, decision_total{content_type}, semantic_hits_total; дашборды Grafana, алерты на рост BLOCK/WARN.

Глава 11

Тестирование

Таблица 11.1: Покрытие тестами

Модуль	Содержание
test_api.py	Health, local moderate, PII, validation 422
test_jailbreak.py	10+ jailbreak/PII/unsafe кейсов
test_semantic_detector.py	Кэш, порог, safe vs jailbreak
benchmark_dataset.json	10 labeled samples для benchmark

Запуск: `pytest -v`. Cloud-тесты требуют API-ключа и опциональны в CI.

Глава 12

Заключение

12.1. Реализовано

- Production-style MVP: FastAPI, два пайплайна, Docker, Swagger.
- Локальный ensemble: toxic-bert, ПИ, regex jailbreak, semantic jailbreak, scorer.
- Облачный контур: ChromaDB RAG, OpenRouter-совместимый LLM, retry, JSON fallback.
- 18 категорий, structured output, benchmark, pytest.

12.2. Направления улучшения

Калибровка порогов на расширенном датасете; гибридный score local+cloud; ONNX/GPU; версионирование политик; Prometheus middleware; мультиязычные эталоны jailbreak.

12.3. Итоговая рекомендация

Для production-контура conversational AI с упором на безопасность, латентность и соответствие 152-ФЗ: **основной путь --- LocalModerator; CloudModerator ---** вторичный слой для спорных случаев и policy-grounded review.